NETWORK PROGRAMMING · TOPIC 1 · HUST

Online Chess – Kế hoạch tuần 21/10 – 27/10/2026

Tuần 1 / 8 · Java thuần, không dùng thư viện luật cờ · Trọng tâm: server quản lý nhiều phòng chơi chạy song song

Nhóm (2 người): Doãn Khang · [Thành viên 2] | **Ngày lập:** 20/10/2026

> **Mục tiêu tuần 21/10:** có khung project chạy được và bản đặc tả chung để hai người code song song từ tuần sau. Cuối tuần (thứ Ba 27/10) phải demo được:
>
> - **2–3 cửa sổ terminal nối vào server bằng `nc localhost 5000`, đăng nhập và thấy nhau trong danh sách online.** Protocol dạng text theo dòng nên test được ngay bằng `nc` / `telnet`, chưa cần client riêng.
> - Khung `Room` / `RoomManager` đã có test chạy nhiều thread.
> - Bộ luật cờ tự viết: đọc/ghi FEN và sinh nước đi cơ bản cho cả 6 loại quân, có test.

A **Doãn Khang**: protocol, server, phòng, matchmaking, Elo B **[Thành viên 2]**: database, luật cờ tự viết, client Swing

Lịch làm việc theo ngày

Ngày	A Server & Room	B Database & Luật cờ
T4 21/10	Họp kickoff 1 giờ với B. Tạo repo GitHub, Maven multi-module `common / server / client`, Java 17, `.gitignore`, README.	Họp kickoff. Cùng A chốt interface `ChessGame` (`tryMove`, `fen`, `turn`, `isOver`, `result`) để A code Room mà không phải chờ. Cài MySQL.
T5 22/10	Viết `PROTOCOL.md` bản 1. Trong `common`: `MessageType`, `Message`, `parse()` / `toLine()`, kiểm tra số trường, giới hạn 1024 ký tự. Unit test.	`schema.sql` (bảng `users`, `games`), `seed.sql` với 5 tài khoản test. Lớp `Color`, `PieceType`, `Piece`, `Square`, `Move`.
T6 23/10	`Server` (accept loop), `ClientHandler` (thread đọc), `ClientWriter` (thread ghi từ `LinkedBlockingQueue`), `Session`.	`Database` (JDBC, đọc cấu hình từ file), `UserDao.findByUsername`, tiện ích hash mật khẩu PBKDF2 kèm test. A cần phần này cho LOGIN vào cuối tuần.
T7–CN 24–25/10	`SessionManager`: LOGIN (gọi UserDao), chặn đăng nhập trùng, LIST_ONLINE, báo cho mọi người khi có người vào/ra, dọn session khi mất kết nối. Test bằng nhiều cửa sổ `nc`.	`Board` (mảng 8×8), đọc/ghi FEN. Sinh nước đi cơ bản cho tốt (đi 1–2 ô, ăn chéo), mã, tượng, xe, hậu, vua. Test cho từng loại quân.
T2 26/10	**Khung Room:** `RoomState`, `RoomType`, `Room` với các hàm `synchronized join / leave / start` theo máy trạng thái (Hình 1, phụ lục); `RoomManager.create / get / remove`. Test JUnit: 10 thread cùng join một phòng → đúng 1 thread vào được.	`isSquareAttacked`, `isInCheck`, lọc bỏ nước đi khiến vua bị chiếu. Review pull request của A.
T3 27/10	**Tích hợp và demo nội bộ** (cả hai): merge vào `main`, chạy toàn bộ test, demo 2–3 cửa sổ `nc` đăng nhập và thấy nhau online. Họp 30 phút: việc tồn đọng, rủi ro, chia việc tuần 28/10.	

Checklist hoàn thành tuần (đến hết 27/10)

✓	Sản phẩm	Người
☐	Repo GitHub với Maven multi-module, `mvn package` chạy được	A
☐	`PROTOCOL.md` bản 1 được cả hai đồng ý; codec message trong `common` có unit test	A
☐	Server nhận nhiều client, mỗi client 1 thread đọc + 1 thread ghi; LOGIN và LIST_ONLINE chạy với MySQL	A
☐	Khung `Room` / `RoomManager` và test race condition đầu tiên xanh	A
☐	`schema.sql`, `seed.sql`; `UserDao` và hash mật khẩu PBKDF2 có test	B
☐	Interface `ChessGame` được chốt (A dùng tạm bản giả để code Room)	A B
☐	`Board` + FEN + sinh nước đi cơ bản cho 6 loại quân + kiểm tra chiếu, có test	B

Lưu ý và việc cần chốt trong tuần

- **Không dùng thư viện luật cờ:** toàn bộ luật tự viết trong `common/chess`. Đây là phần nhiều logic nhất nên B bắt đầu ngay từ tuần 1. Nhập thành, bắt tốt qua đường, phong cấp, chiếu hết, hết nước đi làm ở tuần 2.
- **Cách kiểm tra luật cờ (perft):** đếm số nước đi hợp lệ từ thế khởi đầu. Kết quả đúng phải là 20 (độ sâu 1), 400 (độ sâu 2), 8.902 (độ sâu 3), 197.281 (độ sâu 4). Sai một luật là số lệch ngay.
- **Driver MySQL (`mysql-connector-j`) và JUnit:** đây không phải thư viện luật cờ. Java cần driver này để kết nối MySQL, còn JUnit chỉ dùng để test. Nếu môn cấm cả hai thứ này thì phải hỏi giảng viên.
- **Phiên bản Java:** thống nhất Java 17 (LTS) cho cả hai máy.
- **Quy trình Git:** mỗi người một nhánh `feature/...`, merge vào `main` qua pull request; người còn lại review.
- **Slide đề xuất** vẫn ghi “[Member 2] · [Member 3]”: cần sửa lại thành 2 thành viên.

Phụ lục – Thiết kế tổng thể để tham khảo

Các mục dưới đây là thiết kế chung cho cả 8 tuần. Việc của tuần 21/10 ở trên dựa theo các mục này.

1. Kiến trúc và cấu trúc code

Dùng Maven multi-module, Java thuần. Phần mạng tự viết bằng `java.net.ServerSocket` / `Socket`; không dùng Netty, Spring hay thư viện luật cờ. Luật cờ nằm trong `common` để server dùng làm trọng tài, client dùng để tô sáng các nước đi hợp lệ.

```
chess-online/
├── common/
│   ├── protocol/ MessageType, Message, codec dùng chung cho client và server
│   └── chess/    Board, Piece, Move, MoveGenerator, Fen, ChessGame (luật cờ tự viết)
├── server/
│   ├── net/      Server (accept loop), ClientHandler (reader), ClientWriter (writer)
│   ├── session/  Session, SessionManager (ai đang online, đang ở phòng nào)
│   ├── room/     Room, RoomManager, RoomState, RoomType, Invitation   ← TRỌNG TÂM
│   ├── match/    MatchmakingService (hàng đợi random và skill)
│   ├── rating/   EloCalculator
│   └── db/       UserDao, GameDao, Database (JDBC + MySQL)
└── client/
    ├── net/      Connection (socket + thread lắng nghe)
    └── ui/       LoginFrame, LobbyFrame, GameFrame, BoardPanel (Swing)
```

Mô hình thread ở server

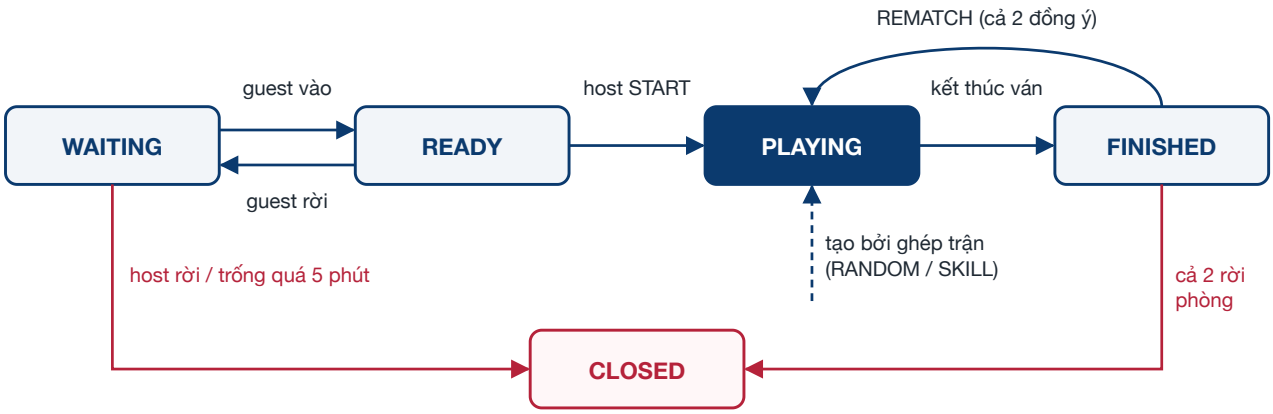
- Mỗi client có **2 thread**. *Reader* đọc lệnh và xử lý. *Writer* lấy message từ một `LinkedBlockingQueue` của client đó rồi ghi ra socket. → Room không bao giờ ghi trực tiếp vào socket, chỉ đẩy message vào hàng đợi. Nhờ vậy một client mạng chậm không làm treo cả phòng.
- Có **1** **`ScheduledExecutorService`** dùng chung cho các việc hẹn giờ: thời gian chờ reconnect, lời mời hết hạn, dọn phòng trống, chạy matchmaking.
- Có **1** **`dbExecutor`** riêng để ghi database, không chạy trên thread đang giữ lock phòng.

2. Thiết kế phòng (phần trọng tâm)

2.1 RoomManager

- `ConcurrentHashMap<Integer, Room> rooms`. Room ID sinh bằng `AtomicInteger`.
- Có 3 cách tạo phòng, cùng dùng chung lớp `Room`:
  - `PRIVATE`: người chơi tự tạo phòng rồi mời người khác.
  - `RANDOM` và `SKILL`: MatchmakingService ghép xong cặp thì tạo phòng.
- Mỗi người chơi **chỉ ở tối đa một phòng**. Ràng buộc này nằm ở `Session.currentRoom`; lệnh nào cũng kiểm tra điều kiện này trước khi chạy.

2.2 Máy trạng thái của Room



Kết thúc ván: chiếu hết · hết nước đi (stalemate) · đầu hàng · hòa · quá thời gian · mất kết nối quá hạn

Hình 1. Các trạng thái của một phòng. Phòng RANDOM/SKILL được tạo thẳng ở PLAYING, không qua WAITING/READY.

Mỗi lệnh chỉ hợp lệ ở một số trạng thái nhất định. Ví dụ `MOVE` chỉ nhận khi đang `PLAYING` và đúng lượt; `INVITE` chỉ nhận khi đang `WAITING`. Lệnh sai trạng thái thì server trả `ERROR`. Đây chính là phần "server từ chối message sai lượt" đã ghi trong slide.

2.3 Dữ liệu của mỗi Room

- `host`, `guest` (Session), `white` / `black`
- `ChessGame game`: thế cờ hiện tại, lưu dưới dạng FEN và danh sách nước đi
- `drawOfferedBy`, `rematchVotes`
- `Map<String, Invitation> pendingInvites`
- `ScheduledFuture<?> disconnectTimer`
- (Mở rộng nếu còn thời gian) `Set<Session> spectators`: người xem. Tính năng này demo "nhiều phòng" rất trực quan.

2.4 Đồng bộ hóa: mỗi phòng một lock

Mọi thao tác thay đổi trạng thái phòng đều dùng `synchronized` trên chính phòng đó. Kết quả:

- Phòng 1 và phòng 2 chạy song song hoàn toàn.
- Hai lệnh vào cùng một phòng (ví dụ hai người cùng bấm "accept draw") được xếp hàng chạy lần lượt.

```
public class Room {
    private RoomState state = RoomState.WAITING;

    public synchronized void handleMove(Session from, String uci) {
        if (state != RoomState.PLAYING)  { from.send(Msg.error("NOT_PLAYING")); return; }
        if (from != sideToMove())        { from.send(Msg.error("NOT_YOUR_TURN")); return; }
        if (!game.tryMove(uci))          { from.send(Msg.moveRejected(uci)); return; }
        drawOfferedBy = null;            // đi quân thì lời đề nghị hòa bị hủy
        broadcast(Msg.moved(id, uci, game.fen(), game.turn()));  // chỉ đẩy vào queue
        if (game.isOver()) finish(game.result(), game.reason());
    }
}
```

Quy tắc tránh deadlock

1. Khi đang giữ lock phòng, không làm I/O mạng và không gọi DB đồng bộ. Ghi DB (Elo, lịch sử ván) thì gửi sang `dbExecutor`.
2. Không bao giờ giữ lock của hai phòng cùng lúc.
3. Thứ tự lấy lock luôn cố định: **Session** → **Room**, không bao giờ theo chiều ngược lại.

2.5 Các tình huống tranh chấp cần xử lý (và viết test)

Tình huống	Cách xử lý
Hai người cùng accept lời mời vào một phòng	Phòng kiểm tra `guest == null` bên trong `synchronized`. Người đến sau nhận `ROOM_FULL`.
Mời người đang ở phòng khác hoặc đang trong hàng đợi	Từ chối, trả `USER_BUSY`. Nếu họ accept lời mời thì rút họ khỏi hàng đợi matchmaking.
Accept lời mời đã hết hạn (30 giây) hoặc của phòng đã đóng	Trả `INVITE_EXPIRED`.
Host rời phòng khi đang `WAITING`	Đóng phòng, hủy toàn bộ lời mời đang chờ.
Cả hai cùng bấm đề nghị hòa	Lệnh đến sau được coi là chấp nhận hòa.
Gửi nước đi sau khi ván kết thúc, hoặc nhầm phòng cũ	Mọi lệnh trong ván đều kèm `roomId`. Server từ chối nếu không khớp phòng hiện tại.
Rớt mạng khi đang `PLAYING`	Đánh dấu DISCONNECTED, báo đối thủ, hẹn giờ 60 giây. Quá hạn thì xử thua.
Reconnect trong 60 giây	Khi LOGIN, SessionManager tìm phòng cũ, gắn session mới vào, hủy hẹn giờ và gửi lại `ROOM_STATE` kèm FEN.
Cả hai cùng rớt mạng	Ai không quay lại kịp thì thua. Nếu cả hai đều quá hạn thì ván bị hủy, không tính Elo.
Một tài khoản đăng nhập hai nơi	Kick session cũ; session mới thay vào chỗ đó, kể cả trong phòng.

3. Protocol: mỗi dòng một message, các trường cách nhau bằng `|`

Encoding UTF-8. Username giới hạn `[A-Za-z0-9_]{3,16}` nên không cần escape ký tự. Server bỏ qua dòng dài quá 1024 ký tự. Nước đi theo ký hiệu **UCI** (`e2e4`, `e7e8q`); thế cờ gửi dạng **FEN**.

Client → Server	Server → Client
`LOGIN|user|pass|RANDOM` hoặc `SKILL`	`LOGIN_OK|rating` · `LOGIN_FAIL|reason`
`LIST_ONLINE` · `LIST_ROOMS`	`ONLINE_LIST|u1:1500:IDLE,u2:1420:IN_GAME` · `ROOM_LIST|...`
`CREATE_ROOM`	`ROOM_CREATED|roomId`
`INVITE|roomId|user`	gửi người được mời: `INVITED|inviteId|from|roomId`
`ACCEPT_INVITE|inviteId` · `DECLINE_INVITE|inviteId`	`ROOM_STATE|roomId|state|white|black|fen|turn`
`LEAVE_ROOM|roomId` · `START|roomId`	`GAME_START|roomId|color|opponent|oppRating`
`FIND_MATCH` · `CANCEL_MATCH`	`MATCH_FOUND|roomId`
`MOVE|roomId|e2e4`	gửi cả hai: `MOVED|roomId|e2e4|fen|turn` · gửi người đi: `MOVE_REJECTED|reason`

`OFFER_DRAW|roomId` · `ACCEPT_DRAW` · `DECLINE_DRAW` · `RESIGN`	`DRAW_OFFERED` · `DRAW_DECLINED`
	`GAME_OVER|roomId|WHITE_WIN|CHECKMATE|+12`
`LEADERBOARD` · `PING`	`LEADERBOARD|...` · `PONG` · `ERROR|code|msg`

Bảng này nên được viết thành file `PROTOCOL.md` ngay tuần đầu để cả hai người code theo cùng một bản đặc tả.

4. Các phần còn lại

- **Luật cờ (tự viết, không dùng thư viện):**
  - `Board`: mảng 8×8, lượt đi, quyền nhập thành, ô bắt tốt qua đường, số nửa nước (luật 50 nước).
  - `MoveGenerator`: sinh nước đi theo từng loại quân, sau đó lọc bỏ nước khiến vua của mình bị chiếu.
  - Luật đặc biệt: nhập thành (vua không bị chiếu, không đi qua ô bị khống chế), bắt tốt qua đường, phong cấp.
  - Kết thúc ván: chiếu hết, hết nước đi (stalemate); nếu còn thời gian thì thêm luật 50 nước và không đủ quân chiếu hết.
  - `Fen`: đọc/ghi FEN. Nước đi nhận theo ký hiệu UCI (`e2e4`, `e7e8q`).
  - Test: mỗi luật một test JUnit, cộng với perft (20 / 400 / 8.902 / 197.281 ở độ sâu 1–4).
  - Room chỉ dùng interface `ChessGame`, nên A code được Room bằng bản giả trong lúc B viết luật.
- **Matchmaking:**
  - Random: hàng đợi FIFO, đủ 2 người là ghép.
  - Skill: hàng đợi theo rating. Ban đầu chỉ ghép chênh lệch ±100 điểm, mỗi 10 giây chờ thì nới thêm 50 điểm.
  - Tác vụ chạy mỗi giây quét hàng đợi rồi gọi `RoomManager.createMatchRoom(a, b, type)`.
- **Elo:** `E = 1 / (1 + 10^((Rb − Ra)/400))`, `R' = R + K·(S − E)`, K = 32, rating khởi điểm 1200. Chỉ server tính; client chỉ hiển thị.
- **Database MySQL:**
  - `users(id, username UNIQUE, password_hash, salt, rating, wins, losses, draws, created_at)`
  - `games(id, room_type, white_id, black_id, result, end_reason, moves TEXT, white_delta, black_delta, started_at, ended_at)`
  - Mật khẩu hash bằng PBKDF2 (có sẵn trong `javax.crypto`), không lưu plain text.
  - Cập nhật Elo và insert vào `games` trong **cùng một transaction**.
- **Client Swing:**
  - Thread mạng nhận message rồi chuyển sang `SwingUtilities.invokeLater` để cập nhật giao diện.
  - Lobby: danh sách online, danh sách phòng, hộp thoại lời mời, nút Find match.
  - GameFrame: bàn cờ (vẽ lại theo FEN server gửi), lịch sử nước đi, các nút Hòa / Đầu hàng / Rời phòng.

5. Lộ trình 8 tuần (điều chỉnh theo deadline của môn)

Tuần	Việc	Kết quả kiểm tra được
1 · 21/10–27/10	**A** **Khung project, `PROTOCOL.md`, server, LOGIN, LIST_ONLINE, khung Room.** **B** **Schema DB, UserDao, Board, FEN, nước đi cơ bản** (chi tiết ở đầu tài liệu).	2–3 cửa sổ `nc` đăng nhập và thấy nhau online

2 · 28/10–03/11	**A** Room qua mạng: CREATE / INVITE / ACCEPT / DECLINE / LEAVE / LIST_ROOMS, hết hạn lời mời. **B** Nhập thành, bắt tốt qua đường, phong cấp, chiếu hết, hết nước đi; perft đúng.	Nhiều phòng tồn tại cùng lúc; luật cờ đủ
3 · 04/11–10/11	**A** Máy trạng thái đầy đủ, test race condition cho mọi tình huống ở mục 2.5; gắn ChessGame thật vào Room: MOVE, GAME_OVER, hòa, đầu hàng. **B** `Connection`, LoginFrame, LobbyFrame.	Chơi trọn một ván qua `nc`
4 · 11/11–17/11	**A** MatchmakingService (random và skill) tạo phòng. **B** BoardPanel, GameFrame.	Chơi trọn một ván qua giao diện
5 · 18/11–24/11	**A** Elo, GameDao, cập nhật trong một transaction, lệnh LEADERBOARD. **B** danh sách phòng, hộp thoại lời mời, nút Find match / Hòa / Đầu hàng.	Ghép trận tự động, rating đổi sau mỗi ván
6 · 25/11–01/12	**A** Rớt mạng và reconnect, kick khi đăng nhập trùng, dọn phòng trống. **B** màn hình bảng xếp hạng, giao diện khi mất kết nối.	Đủ 10 yêu cầu của đề
7 · 02/12–08/12	**A** **Stress test bằng bot** (mục 6), sửa lỗi đồng bộ. **B** chạy thử toàn bộ luồng trên giao diện, sửa lỗi.	50 phòng chạy song song không deadlock
8 · 09/12–15/12	Cả hai: sửa bug, viết báo cáo (sơ đồ trạng thái, sơ đồ thread, protocol, luật cờ), chuẩn bị demo.	Nộp bài

6. Cách chứng minh phần “nhiều phòng” chạy đúng

1. **Bot client không giao diện (`BotClient`):** chạy 100 bot (50 phòng), mỗi bot đi một nước hợp lệ ngẫu nhiên sau mỗi 50–200 ms. Kiểm tra: không deadlock (dùng `jstack` khi nghi treo), không ván nào lẫn nước đi của phòng khác, số ván trong bảng `games` đúng bằng số ván đã kết thúc.
2. **Test race condition bằng JUnit:** dùng `CountDownLatch` cho 10 thread cùng accept vào một phòng; kết quả phải là đúng 1 thread vào được.
3. **Demo trên lớp:** mở 4–6 client, chạy 2–3 phòng song song (một phòng tạo bằng lời mời, một phòng ghép random, một phòng ghép theo skill), ngắt mạng một client để demo reconnect.

Ghi chú: lộ trình 8 tuần là ước lượng, cần chỉnh lại theo deadline nộp bài thật của môn. Mốc tuần 21/10 vẫn giữ nguyên dù deadline là khi nào.